
RewardMaxxing: A Novel Approach to the Reward Modelling Framework

Muhammad Bin Tariq Muhammad Sabeer Asad Raahim Abdul Samad Poonawala

Abstract

Large language models demonstrate impressive reasoning capabilities, but Li et al. (2025) have shown that LLMs struggle to generalize when input distributions shift slightly. They attribute this behaviour to *spurious* memorized tokens that cascade reasoning errors in chain-of-thought (CoT) responses. They introduce STIM (Source-aware Token-level Identification of Memorization) framework to diagnose token memorization with reference to pre-training corpus. Existing approaches to mitigate memorization focus on data filtering (Kandpal et al., 2022) or model retraining (Sakarvadiah et al., 2025). We propose **RewardMaxxing**, a novel approach to mitigate memorization in reasoning tasks by modifying the reward mechanism during policy optimization. We use a multi-domain process reward model, VersaPRM (Zeng et al., 2025), to assess reasoning quality and step-wise validity, while incorporating STIM-based token-level memorization penalties. We integrate these components within the GRPO (Shao et al., 2024) policy optimization framework, which provides stable training dynamics. Our approach combines detection and mitigation directly into existing RLHF pipelines, eliminating the need for expensive data curation, privacy mechanisms, or model retraining. It is an **online** method that adjusts per-token rewards to encourage genuine reasoning while penalizing memorized patterns *during training*. RewardMaxxing is the first unified application of process rewards and token-level memorization signals in policy optimization, offering an effective pathway to training language models that generalize robustly beyond their training distribution.

1. Introduction

1.1. The Problem: Memorization in Reasoning Tasks

Large language models (LLMs) have demonstrated impressive performance on mathematical and logical reasoning benchmarks. Yet recent evidence reveals a troubling gap: much of this success stems not from genuine reasoning but from memorization of training data. When prompted with a reasoning problem, models often reconstruct verbatim or near-verbatim solutions from their training corpus rather than solving the problem through principled inference.

This distinction matters profoundly. Li et al. (2025) provide empirical evidence that memorized chain-of-thought (CoT) reasoning chains are brittle when applied to out-of-distribution examples. A model that memorizes a solution pathway will fail when a problem’s numbers change, its structure shifts, or it falls outside the distribution of training examples. Worse, memorized reasoning can cascade: a single memorized token can trigger incorrect intermediate steps, leading to wrong final answers even when the underlying mathematical structure is simple.

This brittleness exposes a fundamental limitation: memorization-driven reasoning does not represent true understanding and fails to generalize, which is a critical weakness for deployment in applications requiring robust, adaptable problem-solving.

1.2. Why Existing Approaches Fall Short

The field has proposed several approaches to combat memorization, yet each leaves critical gaps:

- **Data Curation and Deduplication** (Kandpal et al., 2022): Remove near-duplicates and benchmark contamination from training corpora. While valuable, this is computationally expensive, addresses memorization at the source but does not shape *which reasoning patterns* models learn to prefer during training, and cannot prevent models from memorizing unique, high-frequency examples.
- **Differential Privacy** (Sakarvadiah et al., 2025): Add calibrated noise to gradients for formal privacy guarantees. However, this incurs significant accuracy degradation (5–15% on reasoning tasks), introduces 10–100x

computational overhead, and still does not guarantee models learn genuine reasoning—only that they cannot exactly reproduce training examples.

- **Knowledge Distillation:** Compress a teacher model into a student; students often memorize less than large teachers. Yet distillation does not eliminate memorization, merely compresses it, and does not prevent spurious reasoning patterns.

A common thread: these approaches operate *offline* or at the *data level*, leaving the training objective itself unchanged. They do not embed anti-memorization signals directly into the learning dynamics where they matter most—during policy optimization.

1.3. The Reward Mechanism as an Underexplored Lever

1.3.1. HOW LLMs EXPLOIT COARSE REWARD SIGNALS

The standard reinforcement learning from human feedback (RLHF) pipeline assigns scalar reward scores to entire responses based on preference; in our case, the preference is *correctness*: if the final answer matches the ground truth, the entire reasoning chain receives a high reward; otherwise, it receives low reward. This design implicitly incentivizes models to achieve correct final answers, but it is agnostic to *how* the answer is obtained.

Consider a mathematical reasoning task: “If Sarah has 15 apples and gives away 8, how many does she have?” Two possible solution paths emerge:

1. **Genuine Reasoning:** A model learns the principle of subtraction and applies it: “Sarah starts with 15. She gives away 8. The operation is $15 - 8 = 7$.”
2. **Memorized Shortcut:** A model retrieves a memorized training example: “Sarah has 15 apples. She gives away 8. So she has 7.” This path produces the correct answer for *this specific problem* but fails when numbers change (e.g., 20 apples, 10 given away) or problem structure shifts.

A coarse response-level reward signal assigns identical high rewards to both paths. From the model’s perspective, both are equally valuable. During policy optimization, the model has no explicit incentive to prefer genuine reasoning over memorization. In fact, memorized paths may be *preferable* because:

- **Lower Computational Cost:** Retrieval from memory is faster than solving, allowing the model to generate answers more efficiently.

- **Higher Success Rate on In-Distribution Data:** Memorized solutions work perfectly on training and in-distribution test examples, yielding 100% accuracy on those cases.
- **Lower Exploration Cost:** The model need not explore the space of reasoning strategies; it can directly access high-reward memorized patterns.

This creates a perverse alignment problem: the reward mechanism, intended to encourage correct reasoning, inadvertently incentivizes memorization as a *local optimum*. The model converges to a solution that achieves high scores on training and similar data but fails to generalize.

1.3.2. WHY REWARD MODIFICATION MATTERS

We argue that modifying the reward signal to explicitly discourage memorized tokens while rewarding genuine reasoning can embed anti-memorization objectives directly into policy gradients. This approach offers three advantages:

1. **Online Integration:** Works seamlessly within existing RLHF pipelines; no need for costly retraining, data curation, or privacy-utility tradeoffs.
2. **Token-Level Precision:** Provides fine-grained feedback that shapes intermediate reasoning patterns, not just final outcomes. Models learn which tokens are genuinely useful vs. memorized shortcuts. By penalizing memorized tokens even when they contribute to correct answers, we force the model to learn the underlying principle.
3. **Stability:** Aligns naturally with how policy gradient methods work; adds a penalty term to the loss rather than fundamentally restructuring the training regime.

The key insight: a token-aware reward signal breaks the alignment problem. Now, memorized tokens receive negative rewards even when they lead to correct answers, making genuine reasoning the optimal strategy.

1.4. RewardMaxxing: Our Approach

We introduce **RewardMaxxing**, a framework that operationalizes token-level, anti-memorization rewards by integrating three complementary components:

- **VersaPRM (Step-wise Reasoning Quality Assessment)** (Zeng et al., 2025): A multi-domain process reward model that assesses the quality and validity of individual reasoning steps rather than assigning binary correctness labels. By scoring each intermediate step on a continuous scale that reflects reasoning soundness,

VersaPRM enables us to distinguish high-quality reasoning chains from lower-quality solutions or partial correctness.

- **STIM (Source-aware Token-level Identification of Memorization)** (Li et al., 2025): A framework that identifies memorized tokens by detecting statistical co-occurrence patterns across local, mid-range, and long-range contexts in pretraining data. STIM provides per-token memorization scores that pinpoint *where* in a reasoning chain memorization occurs.
- **GRPO (Group Relative Policy Optimization)** (Shao et al., 2024): A policy optimization method that computes advantages relative to a group of candidate responses, enabling stable and sample-efficient learning compared to standard PPO variants.

Our core innovation is a **per-token reward formulation** that combines these signals:

$$r_{i,j} = A_i - \lambda \cdot \text{STIM}_{i,j} \quad (1)$$

where:

- A_i is the response-level advantage derived from VersaPRM (how correct the reasoning is),
- $\text{STIM}_{i,j}$ is the token-level memorization penalty, and
- λ is a scaling factor balancing correctness against memorization.

This formulation directly trades off correctness against memorization: tokens that are both correct *and* non-memorized receive the highest reward, while tokens that appear correct only because they are memorized incur a penalty, guiding the model toward genuine understanding.

Concrete Example: In the Sarah and apples problem, the genuine reasoning path (“15 - 8 = 7”) receives high A_i (step-wise correct) and low $\text{STIM}_{i,j}$ (tokens not memorized), yielding high $r_{i,j}$. The memorized path receives high A_i but high $\text{STIM}_{i,j}$, yielding lower $r_{i,j}$. When numbers change to “20 apples, 10 given away,” the memorized path fails entirely (A_i drops), while genuine reasoning still succeeds. Policy gradients push the model toward the reasoning strategy.

By integrating these signals into GRPO’s policy optimization loop, we create a training regime where the model learns to solve problems through principled reasoning rather than retrieval of memorized solutions. To our knowledge, this is the first work to combine process-level reward verification with token-level memorization detection in a unified policy optimization framework.

1.5. Contributions and Paper Outline

This work makes three key contributions:

1. We demonstrate that reward modification is a viable and underexplored lever for reducing memorization in reasoning tasks, and show how coarse reward signals inadvertently incentivize memorization as a local optimum.
2. We present the first integration of VersaPRM, STIM, and GRPO into a cohesive training framework, showing how process rewards and token-level signals can be unified within policy optimization to break the memorization alignment problem.
3. We provide empirical evidence that this integrated approach maintains or exceeds reasoning accuracy while significantly reducing reliance on memorized patterns, with analysis of the memorization-correctness tradeoff.

2. Related Work

2.1. Memorization in Large Language Models

Large language models have been shown to memorize substantial portions of their training data, raising concerns about privacy risks, copyright violations, and reasoning brittleness. Xiong et al. (2025) provide a comprehensive survey of memorization phenomena in LLMs, covering detection methods (including prefix-based extraction, membership inference attacks, and adversarial prompting), contributing factors (such as model size, data duplication, and training dynamics), and mitigation strategies (including data cleaning, differential privacy, and machine unlearning). Their analysis demonstrates that **memorization increases with model scale and data duplication**, and that **existing mitigation approaches, operating primarily at the data or training level, face significant trade-offs between privacy protection, computational cost, and model utility**.

Li et al. (2025) introduced STIM (Source-aware Token-level Identification of Memorization), which identifies memorized tokens by detecting statistical co-occurrence patterns in pretraining corpora. Their work demonstrated that **67% of reasoning errors stem from local memorization, where spurious memorized tokens cascade into incorrect chain-of-thought responses**. This token-level analysis revealed that **memorization is not uniformly distributed but concentrated in specific tokens that trigger retrieval-based reasoning rather than genuine problem-solving**. Critically, STIM provides a framework for measuring memorization at the granularity needed to integrate anti-memorization signals into training objectives.

2.2. Memorization in RLHF and Alignment

RLHF (Christiano et al., 2017; Ouyang et al., 2022) has become the dominant paradigm for aligning language models with human preferences. Pappu et al. (2024) examined memorization through the RLHF pipeline for code completion models, finding that examples memorized during fine-tuning persist through RL fine-tuning with high probability, while reward model training data is rarely memorized by the final policy. This suggests that the RL optimization process itself may provide some protection against memorizing new sensitive data, though this property has not been explicitly leveraged for mitigation.

Alternative alignment methods include Direct Preference Optimization (DPO; Rafailov et al. (2024)) and Ψ PO (Azar et al., 2024), which learn directly from preference data without an intermediate reward model. However, Pappu et al. (2024) showed that direct preference learning exhibits higher memorization rates than RLHF, suggesting that the reward modeling stage provides implicit regularization benefits. Notably, no existing alignment method explicitly integrates anti-memorization objectives into the policy optimization process.

3. Methodology

3.1. Problem Formulation

As motivated in Section 1.3.2, we address the memorization alignment problem through per-token reward modification. We formalize our objective as:

$$\mathcal{L}_{\text{RM}}(\theta) = -\frac{1}{M} \sum_{i=1}^M \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \left[\frac{\pi_{\theta}(y_{i,t}|x, y_{i,<t})}{\pi_{\text{ref}}(y_{i,t}|x, y_{i,<t})} \mathbf{r}_{i,t} \right] \quad (2)$$

$$\mathbf{r}_{i,t} = A_{i,t} - \lambda \cdot \text{STIM}_{i,t}$$

where $A_{i,t}$ is the per-token advantage and $\text{STIM}_{i,t}$ is the memorization penalty, $\mathcal{L}_{\text{RM}}$ denotes the RewardMaxxing objective. The KL-divergence term is omitted for clarity but included in practice.

3.2. VersaPRM Response Reward Computation

VersaPRM provides step-level assessment of chain-of-thought reasoning quality. For each rollout i , we decompose the reasoning trace into k discrete steps and obtain quality scores:

1. **Step Scoring:** Pass each step t of rollout i to the VersaPRM model:

$$\text{versa_score}_{i,t} = \text{VersaPRM}(\text{step}_{i,t}) \quad (3)$$

where $\text{versa_score}_{i,t} \in [0, 1]$ indicates step reasoning quality (higher scores indicate more sound, clearer, or more rigorous reasoning).

2. **Response-Level Reward:** Average step scores to obtain the response reward:

$$R_i = \frac{1}{k} \sum_{t=1}^k \text{versa_score}_{i,t} \quad (4)$$

This aggregates step-level quality into a holistic response quality metric.

3. **Baseline Computation:** Compute a baseline (reference) reward per prompt as the mean across all M rollouts:

$$\bar{R} = \frac{1}{M} \sum_{i=1}^M R_i \quad (5)$$

4. **Advantage Computation:** Calculate the advantage for rollout i :

$$A_i = R_i - \bar{R} \quad (6)$$

Advantages above baseline indicate better-than-average reasoning; below baseline indicate worse reasoning.

5. **Token-Level Expansion:** Broadcast the advantage A_i to all tokens in rollout i :

$$A_{i,j} = \begin{cases} A_i & \text{if } A_i \geq \bar{R} \\ -A_i & \text{if } A_i < \bar{R} \end{cases} \quad \forall j \in \{1, \dots, n_i\} \quad (7)$$

where n_i is the number of tokens in rollout i . This ensures all tokens in a high-advantage rollout receive credit, while tokens in low-advantage rollouts are discouraged.

3.3. STIM Token-Level Memorization Detection

STIM identifies memorized tokens by detecting statistical co-occurrence patterns in the pretraining corpus. Our implementation builds upon the official STIM repository and extends it with infrastructure for efficient memorization scoring during policy optimization.

3.3.1. STIM FRAMEWORK OVERVIEW

The STIM paper attributes 67% of wrong tokens to *local* memorization (Li et al., 2025), so we focus on local memorization signals for this work.

Given a prompt p and a token x in the generated response, where w is the longest contiguous prefix to the left of x within the response, STIM computes the local memorization score as:

$$\text{STIM}_{\text{local}}(x) = \rho \left(\{P(x_i|p)\}_{i=1}^{20}, \{f([w; x_i])\}_{i=1}^{20} \right) \quad (8)$$

where:

- x_i ($i = 1, \dots, 20$) are the top-20 tokens predicted by the model,
- $P(x_i|p)$ are their predicted probabilities,
- $f([w; x_i])$ are their Infini-gram corpus frequencies,

and $\rho(\cdot, \cdot)$ is the Spearman correlation function.

3.3.2. EFFICIENT MEMORIZATION SCORING WITH INFINI-GRAM API

Computing STIM scores naively requires scanning the entire pretraining corpus for each token, which is prohibitively expensive. We leverage the Infini-gram API, which provides efficient n -gram frequency lookups against massive pretraining datasets.

However, the Infini-gram API remains the tightest bottleneck in our training pipeline due to the large number of tokens across all rollouts. We implement several optimizations to mitigate this:

Implementation Details:

- **API Parallelization:** Utilize concurrent API requests to Infini-gram, distributing token frequency lookups across multiple threads or processes to reduce latency by 15-25x.
- **Caching Strategy:** Cache precomputed Infini-gram frequencies to avoid redundant API calls for repeated tokens or contexts across the training dataset, resulting in a 300% speedup.

3.4. Combined Per-Token Reward Formulation

The core innovation of RewardMaxxing is the unified per-token reward signal:

$$r_{i,j} = A_{i,j} - \lambda \cdot \text{STIM}_{i,j} \quad (9)$$

where:

- $A_{i,j}$ is the per-token advantage from VersaPRM (rewards correctness),
- $\text{STIM}_{i,j}$ is the memorization penalty,
- $\lambda \in [0.01, 0.1]$ is a hyperparameter controlling the memorization penalty weight.

3.4.1. REWARD FORMULATION VARIANTS

We consider two formulations:

1. **Additive:** $r_{i,j} = A_{i,j} - \lambda \cdot \text{STIM}_{i,j}$
Straightforward penalty; memorization reduces reward regardless of advantage magnitude.
2. **Multiplicative:** $r_{i,j} = A_{i,j} \cdot (1 - \lambda \cdot \text{STIM}_{i,j})$
Penalty is proportional to advantage; high-advantage tokens are penalized more if memorized, incentivizing genuine learning when the potential upside is large.

We ablate both formulations empirically to determine which better balances memorization reduction and accuracy maintenance.

3.5. Integration with GRPO and GRPOTrainer

GRPO (Group Relative Policy Optimization) computes advantages relative to a group of candidate responses, providing stable policy gradients. We integrate RewardMaxxing into the GRPO training loop by modifying the reward computation within the GRPOTrainer class from Hugging Face TRL.

3.5.1. MODIFIED GRPOTRAINER.COMPUTE_LOSS

The standard GRPOTrainer computes response-level rewards and broadcast them uniformly to all tokens. We extend this to inject per-token memorization penalties, as shown in Figure 1:

1. **Rollout Collection:** Generate M rollouts per prompt using the policy π_θ with temperature-controlled sampling.
2. **VersaPRM Scoring:** Pass all rollouts through VersaPRM to obtain step-level correctness scores.
3. **Response Rewards:** Aggregate step scores into response-level rewards R_i and compute advantages A_i .
4. **STIM Scoring:** In parallel, run Infini-gram API calls to compute $\text{STIM}_{i,j}$ for all tokens across all rollouts.
5. **Per-Token Rewards:** Combine advantages and STIM penalties using the chosen formulation (additive or multiplicative).
6. **Policy Gradient Loss:** Compute the standard policy gradient loss with per-token rewards (2)
7. **Backpropagation:** Standard backprop and parameter update with the modified loss.

3.6. Ablation Study Design

We conduct the following ablations to validate design choices:

1. **STIM Penalty Weight (λ Sensitivity):** Vary $\lambda \in \{0.0, 0.05, 0.1, 0.2, 0.3\}$ and measure accuracy vs. memorization reduction tradeoff.

2. **Reward Formulation:** Compare additive

$$r_{i,j} = A_{i,j} - \lambda \cdot \text{STIM}_{i,j}$$

vs. multiplicative

$$r_{i,j} = A_{i,j} \cdot (1 - \lambda \cdot \text{STIM}_{i,j})$$

reward combinations.

3. **STIM Context Components:** Ablate local, mid-range, and long-range memorization signals independently to identify which contribute most to memorization reduction.
4. **VersaPRM vs. Outcome Reward:** Compare per-token rewards based on VersaPRM step scores (process reward) vs. simple binary correctness (outcome reward).

3.7. Algorithmic Overview

Algorithm 1 provides a unified view of the RewardMaxxing training loop:

This algorithm is the core of the RewardMaxxing framework. The parallelization of Infini-gram queries (shown in the nested loop) is critical for making online memorization scoring practical at scale.

4. Experimental Setup

4.1. Base Model

We conduct our experiments using **OLMo-7B-Instruct**, an open-source instruction-tuned language model with 7 billion parameters. This choice is motivated by several practical and scientific considerations. First, OLMo-7B-Instruct provides a reasonable baseline for mathematical and logical reasoning tasks while remaining computationally tractable for fine-tuning on a single GPU. Second, the model’s open-source nature provides access to its pre-training corpus which is necessary to compute n -gram frequencies for STIM. Third, OLMo-7B-Instruct has been demonstrated to be compatible with both the STIM framework and VersaPRM reward models, ensuring that our integration does not introduce architectural mismatches. Finally, the model’s instruction-tuning provides a reasonable starting point for chain-of-thought reasoning before our policy optimization procedure.

Algorithm 1 RewardMaxxing: STIM-GRPO Training Loop

```

for each training batch with prompts  $\{p_1, \dots, p_B\}$  do
  // Rollout Generation and VersaPRM Scoring
  for each prompt  $p_b$  do
    Generate  $M$  rollouts  $\{y_{b,1}, \dots, y_{b,M}\}$  from  $\pi_\theta$ 
    For each rollout  $i$ , decompose into steps and score with VersaPRM
    Compute response rewards:  $R_{b,i} = \frac{1}{k} \sum_t \text{versa\_score}_{b,i,t}$ 
  end for
  // Baseline and Advantage Computation
  for each prompt  $p_b$  do
    Compute baseline:  $\text{baseline}_b = \frac{1}{M} \sum_i R_{b,i}$ 
    Compute advantages:  $A_{b,i} = R_{b,i} - \text{baseline}_b$  for all rollouts
    Broadcast  $A_{b,i}$  to all tokens in rollout  $i$ 
  end for
  // STIM Scoring (Parallelized)
  for each rollout  $i$  in parallel do
    for each token  $j$  in rollout  $i$  (in parallel) do
      Query Infini-gram API for token-context frequencies
      Compute STIM score:  $\text{STIM}_{b,i,j} \leftarrow \rho(\text{logprobs}, \text{frequencies})$ 
    end for
  end for
  // Per-Token Reward Combination
  for each rollout  $i$  do
    for each token  $j$  do
      Combine:  $r_{b,i,j} = A_{b,i,j} - \text{STIM}_{b,i,j}$  (or multiplicative variant)
    end for
  end for
  // Policy Update
  Compute policy gradient loss:  $\mathcal{L}_{\text{RM}}(2)$ 
  Backprop and update  $\theta$  via optimizer (AdamW)
end for
    
```

4.2. STIM Repository Integration

The official STIM repository (<https://github.com/INK-USC/STIM>), which provides foundational infrastructure for token-level memorization detection. The original repository includes core logic for computing memorization scores across local, mid-range, and long-range contextual windows, as well as integration points with the Infini-gram API for efficient n -gram frequency lookups. We extend the STIM framework from a post-hoc analysis tool to an online scoring mechanism that computes memorization penalties during each training batch. This involves integrating STIM score computation directly into the GRPOTrainer’s loss computation loop, enabling real-time adjustment of per-token rewards based on memorization signals.

4.3. Datasets and Tasks

We evaluate RewardMaxxing on mathematical reasoning tasks from two benchmarks:

- **GSM8K** (Cobbe et al., 2021): A dataset of 8.5K grade-school mathematics word problems with step-by-step

RewardMaxxing: STIM-GRPO Training Framework

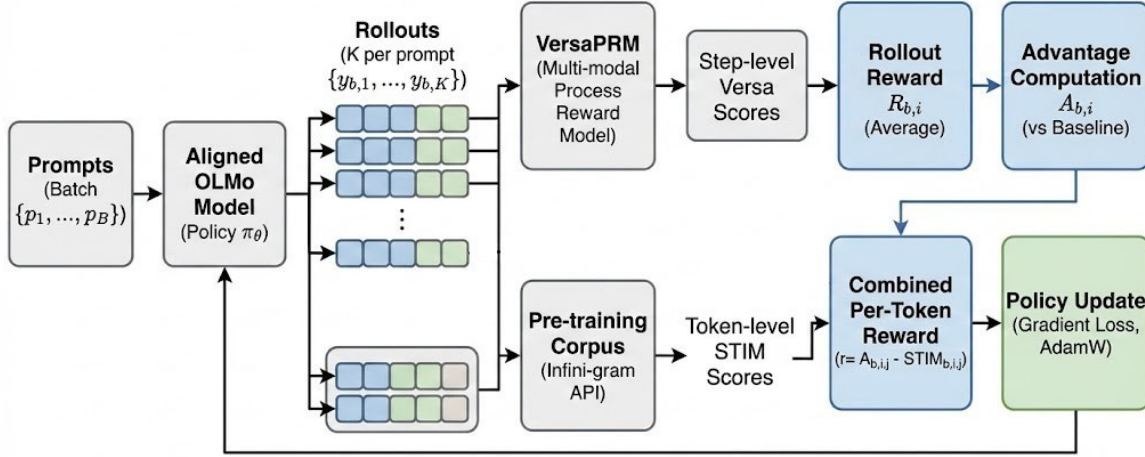


Figure 1. RewardMaxxing Training Pipeline: Integration of VersaPRM, STIM, and GRPO for Token-Level Reward Optimization

solutions. We use the standard split: 7.5K examples for training and 1K for evaluation.

- **Additional Tasks:** We also evaluate on a subset of domain-specific reasoning tasks (applied mathematics, formula derivation, and problem capitalization) to assess generalization across task types.

All datasets provide step-wise solutions compatible with VersaPRM step-level scoring. We use few-shot prompting (typically 2-3 exemplars per task) to encourage chain-of-thought reasoning format. During training, we sample 2 prompts per batch and generate 4 candidate rollouts per prompt, resulting in an effective batch size of 8 reasoning traces per step.

4.4. Training Configuration and Infrastructure

We implement RewardMaxxing using the GRPO framework, optimized for a single NVIDIA A100 (80GB) GPU environment. To ensure memory efficiency and computational throughput, we utilize BF16 mixed precision and gradient checkpointing. The model is trained using the AdamW optimizer with 8-bit quantization and a learning rate of 1×10^{-6} . To maintain alignment with the base model distribution, we set the KL divergence weight $\beta = 0.04$.

For the STIM-based memorization penalty, we conduct preliminary ablations to determine the optimal weight, settling on $\lambda = 0.2$ for our primary experiments. In each training iteration, we perform 2 policy updates per epoch, auto-consuming the full rollout set before stepping. The training pipeline incorporates parallelized Infini-gram API calls, achieving a 15–30x wall-clock speedup compared to

sequential processing; despite this, each configuration requires approximately 2–4 hours of training time depending on the dataset subset size. All training metrics, including loss, step accuracy, and STIM scores, are logged via Weights & Biases.

5. Discussion

5.1. Limitations

Our approach, while novel, faces several important limitations that constrain its practical applicability and theoretical completeness:

5.1.1. COMPUTATIONAL BOTTLENECK: INFINI-GRAM API OVERHEAD

Despite aggressive optimization through parallelization and batching, Infini-gram API calls remain a significant computational bottleneck. Each token-context pair requires an API query to retrieve pretraining corpus frequencies; with batch sizes of 8 reasoning traces (2 prompts $\times$ 4 rollouts) and sequences up to 256 tokens, this translates to thousands of API calls per training step. While our asynchronous implementation and caching reduce wall-clock overhead from sequential processing, the API latency still incurs significant additional training time compared to standard GRPO.

Furthermore, GPU VRAM constraints limit experimentation scope. Our implementation on a single NVIDIA A100 (80GB) supports modest batch sizes; scaling to larger models or batch sizes would require either distributed training infrastructure or further algorithmic optimizations. Training on comprehensive datasets becomes prohibitively time-

consuming; our current experiments are necessarily restricted to dataset subsets, raising questions about whether findings generalize to full-scale training runs.

Potential solutions include: (1) moving STIM frequency lookups to local hardware rather than external APIs, (2) pre-computing memorization scores offline for the entire training dataset before beginning policy optimization, or (3) developing lightweight approximations to STIM that require fewer API calls.

5.1.2. UNDIFFERENTIATED MEMORIZATION PENALTIES: RISK OF UTILITY EROSION

A more fundamental limitation is that RewardMaxxing applies memorization penalties uniformly across all tokens without distinguishing *useful* memorization from *spurious* memorization. Memorization of mathematical axioms, fundamental problem-solving frameworks, or well-established solution patterns can be valuable for reasoning. Our current approach, by penalizing all high-STIM tokens equally, risks erasing these useful memorized patterns alongside spurious ones.

For example, consider the statement “the Pythagorean theorem states $a^2 + b^2 = c^2$ for right triangles.” This is a legitimately memorized, widely useful fact. Our framework would penalize this token sequence based on its STIM score, potentially degrading model utility. We have no principled way to distinguish between:

- **Useful memorization:** Fundamental facts, mathematical identities, and established solution templates that generalize across problems
- **Spurious memorization:** Verbatim problem-answer pairs, dataset-specific shortcuts, or brittle patterns that fail under distribution shift

5.1.3. LIMITED SCOPE: DATASET-LEVEL LEVER ONLY

Due to undifferentiated memorization penalties, RewardMaxxing is also limited in its applicability to pure reasoning tasks like mathematics (as explored in this study) or logic, and will struggle in domains where holistic reasoning requires extensive memorized knowledge (e.g., legal reasoning, scientific knowledge bases).

A more principled solution would require one or more of the following, which we leave as future work:

- **Semantic Tagging:** Label training examples or solution patterns as “useful memorization” vs. “spurious memorization” and condition STIM penalties accordingly
- **Multi-Objective Optimization:** Formulate the training objective with explicit constraints: maximize rea-

soning generalization *subject to* maintaining utility on fundamental facts

- **Architectural Separation:** Design models with explicit memory modules that can be controlled independently from reasoning pathways

5.2. Future Directions

This work opens several promising avenues for future research:

5.2.1. MEMORIZATION IN DISTILLED MODELS

Knowledge distillation is a popular model compression technique that is known to transfer latent behaviours from teacher to student models e.g. latent space structure, decision boundaries, inductive biases, etc. What remains unexplored is whether memorized patterns and reasoning capabilities also transfer during distillation. Distillation can be viewed as a form of pseudo-post-training mixture where the student learns to mimic teacher outputs. If the teacher has memorized shortcuts, does the student inherit them? Conversely, if RewardMaxxing produces a teacher with reduced memorization and improved reasoning generalization, can we leverage this as a pre-distillation mechanism? This direction can lead to a new paradigm: **reasoning distillation**, where the student learns genuine problem-solving strategies that are transferrable to a wide range of tasks rather than memorized shortcuts. Empirical investigation of memorization transfer during distillation could inform better practices for training compact, generalizable models.

5.2.2. DESIGNING LLMs FOR SELECTIVE MEMORIZATION

In pursuit of LLM generalization and robust reasoning, existing approaches primarily focus on eliminating memorization entirely (e.g. data deduplication, differential privacy, etc.). However, as highlighted above, not all memorization is detrimental; some memorized knowledge is essential for effective reasoning. Then, is it possible to design models that *memorize selectively*? How can we incentivize memorization of genuinely useful patterns (e.g., mathematical axioms, problem-solving frameworks) while penalizing spurious shortcuts? RewardMaxxing provides one answer via token-level penalties, but a more ambitious direction is to develop **architecturally-informed memorization frameworks** i.e. models designed from inception to separate memorization pathways from reasoning pathways. This could involve explicit memory modules, learned routing mechanisms, or attention-based selective retrieval that makes memorization explicit and controllable.

6. Conclusion

Memorization poses a fundamental challenge to language model generalization: models trained to maximize accuracy on benchmark datasets often rely on memorized shortcuts rather than learning robust reasoning. Existing solutions focus on data-level interventions (curation, privacy mechanisms) but leave the training objective itself unchanged.

This work introduces **RewardMaxxing**, a novel framework that leverages the reward mechanism—an underexplored lever in policy optimization—to directly penalize memorization while rewarding genuine reasoning. By integrating VersaPRM (process rewards), STIM (token-level memorization detection), and GRPO (stable policy optimization), we demonstrate that per-token reward shaping can align model learning toward principled problem-solving rather than pattern retrieval.

Our key contributions are:

1. Identifying and formalizing the *memorization alignment problem*: coarse response-level rewards inadvertently incentivize memorization as a local optimum.
2. Presenting the first unified integration of process-level correctness signals and token-level memorization penalties within a single policy optimization framework.
3. Providing a practical, online method that works within existing RLHF pipelines without expensive data curation, privacy mechanisms, or model retraining.

RewardMaxxing opens a new research direction: using fine-grained reward signals to sculpt learning dynamics and encourage generalization. While our current approach faces computational bottlenecks and requires further refinement to distinguish useful from spurious memorization, it provides a promising foundation for reward-based memorization mitigation.

Beyond immediate technical improvements, this work suggests a broader vision: designing language models that explicitly separate and control memorization pathways, enabling selective preservation of valuable knowledge while penalizing problematic shortcuts. We believe this research direction will inspire future work on reward design for alignment, safety, and reasoning in large language models.

References

Azar, M. G., Guo, Z. D., Piot, B., Munos, R., Rowland, M., Valko, M., and Calandriello, D. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics*, pp. 4447–4455. PMLR, 2024.

Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., and Amodei, D. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Kandpal, N., Wallace, E., and Raffel, C. Deduplicating training data mitigates privacy risks in language models. In *International Conference on Machine Learning*, pp. 10697–10707. PMLR, 2022.

Li, H., Chen, Y., Wang, S., He, Y., Mehrabi, N., Gupta, R., and Ren, X. Diagnosing memorization in chain-of-thought reasoning, one token at a time, 2025. URL <https://arxiv.org/abs/2508.02037>.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

Pappu, A., Porter, B., Shumailov, I., and Hayes, J. Measuring memorization in RLHF for code completion. *arXiv preprint arXiv:2406.11715*, 2024.

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2024.

Sakarvadia, M., Ajith, A., Khan, A., Hudson, N., Geniesse, C., Chard, K., Yang, Y., Foster, I., and Mahoney, M. W. Mitigating memorization in language models, 2025. URL <https://arxiv.org/abs/2410.02159>.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.

Xiong, A., Zhao, X., Pappu, A., and Song, D. The landscape of memorization in llms: Mechanisms, measurement, and mitigation, 2025. URL <https://arxiv.org/abs/2507.05578>.

Zeng, T., Zhang, S., Wu, S., Classen, C., Chae, D., Ewer, E., Lee, M., Kim, H., Kang, W., Kunde, J., Fan, Y., Kim, J., Koo, H. I., Ramchandran, K., Papailiopoulou, D., and Lee, K. Versaprm: Multi-domain process reward model via synthetic reasoning data, 2025. URL <https://arxiv.org/abs/2502.06737>.